Current Recursion and Tensor-Network Execution in pyAmpliCol

Purpose. This note summarizes the matrix-element generation strategy used by AmpliCol and the corresponding pyAmpliCol implementation. The common idea is to avoid explicit Feynman-diagram enumeration. Instead, matrix elements are assembled from a directed acyclic graph (DAG) of off-shell currents. AmpliCol emits Fortran code that numerically sweeps this current table. pyAmpliCol builds the same logical DAG in Python, lowers bounded pieces to tensor-network expressions handled by Spenso, compiles the resulting parametric scalar/vector expressions with Symbolica evaluators, and can now execute the staged sweep from Rust through the PyO3 runtime `rusticol`.

1 The AmpliCol Current Recursion

For a fixed leading-color ordering, a current is labelled by the external subset it contains, the particle type carried by the off-shell line, and any spin/chirality information. Schematically, if I is a set or ordered interval of external colored legs, a current is computed as

$$\mathcal{J}^{\alpha}_{t,\chi}(I) = P^{\alpha}{}_{\beta}(t, p_I) \sum_{\substack{I=A\cup B \\ A\cap B=\emptyset}} \mathcal{V}^{\beta}{}_{\gamma\delta}(t_A, t_B \to t)\, \mathcal{J}^{\gamma}_{t_A,\chi_A}(A)\, \mathcal{J}^{\delta}_{t_B,\chi_B}(B). \tag{1}$$

Here t is the current type (gluon, quark, auxiliary tensor, etc.), χ is the Weyl chirality where relevant, $p_I = \sum_{i\in I} p_i$, P is the propagator, and $\mathcal{V}$ is the appropriate Feynman-rule tensor. The four-gluon vertex is treated through the auxiliary antisymmetric tensor route, so it is still represented by cubic current combinations.

AmpliCol's optimized mode does not loop over full helicity configurations as independent computations. It weaves helicity information into the current table itself. Each external helicity state is a source current with a unique bit in an `ext_cur`-style bitset. An internal current is identified by

$$K(\mathcal{J}) = \big(t, \chi, \mathrm{bin}(I), \mathrm{ext\_cur}\big), \tag{2}$$

where $\mathrm{bin}(I)$ records the external subset and ext_cur is the union of the source-current bits feeding the current. When a new vertex contribution produces the same key as an existing current, AmpliCol appends that vertex to the existing current rather than creating a duplicate node. After all currents are constructed, `curr2amp` records the two endpoint currents whose contraction gives each retained helicity amplitude.

The generated Fortran library has the same layered structure:

```
subroutine evaluate_amp(p, amps)
  complex(kind=8) :: val_c(1:6, n_cur)
  complex(kind=8) :: int_c(1:6, n_vert)

  call compute_external_currents(pp, val_c)
  do size = 2, n_external_minus_one
    call compute_vertices_size(pp, val_c, int_c)
    call compute_currents_size(pp, val_c, int_c)
  end do
  call compute_amps(amps, val_c)
end subroutine
```

The arrays `val_c` and `int_c` are the numerical current and interaction tables. This is the performance-critical idea that pyAmpliCol keeps: one topological sweep produces all retained amplitudes, and shared parent currents are evaluated once.

2 The pyAmpliCol DAG

pyAmpliCol first builds an explicit Python representation of the same current table. In the generic compiler, a current is identified by a model-driven physics index rather than by a process-family name. The identity used for deduplication is

$$K_{\text{pyAmpliCol}} = \big(\mathrm{particle\_id}, L, H, \chi, S, F, Q, C, P, A\big), \tag{3}$$

where L is the sorted tuple of external labels, H is the AmpliCol-style helicity-source ancestry bitset, χ is chirality, S is any spin-state tag needed by the source or propagator, F is flavour flow, Q is charge flow, C is the leading-color state, P is the momentum mask, and A is an optional auxiliary kind such as the antisymmetric tensor used for the four-gluon vertex. The implementation also carries `ordered_external_labels`. That field is leading-color ordering metadata used by the color engine and amplitude grouping, but it is deliberately not part of equality or hashing. Deduplication is therefore exactly equality of Eq. (3); no rule is allowed to ask whether the process is "Z plus jets" or any other family.

field	**role in the current identity**
`particle_id`	Model/UFO particle id, e.g. 21, 23, 1, -1, or an auxiliary id.
`external_labels` and `external_mask`	Which external legs are contained in the current. The mask is the bit representation of the labels.
`helicity_ancestry`	Bitwise union of the source-current helicity bits feeding this current. This is how helicity configurations are woven into one shared DAG.
`chirality` and `spin_state`	Local spin information, especially for Weyl fermions and massive-vector sources.
`flavour_flow` and `charge_flow`	Quantum-number flow checked locally by model vertices, e.g. $u \to dW^+$ or $e^+e^- \to \gamma/Z$.
`color_state`	Colour-ordered open-line or trace state used to build the amplitude vector. LC, NLC and full colour differ in the final sparse colour contraction over this vector.
`momentum_mask`	Which all-outgoing momenta are summed for propagators and momentum-dependent vertices.
`auxiliary_kind`	Optional tag distinguishing auxiliary tensor currents from ordinary particles.

Source currents are built from external leg metadata. Internal currents are then generated by increasing current size, with all interactions that feed the same current index attached to one node.

```
for external helicity source s:
    add_current(key=(pdg_s, labels_s, chirality_s, bit_s), is_source=True)

for current_size in increasing_sizes:
    for compatible split A | B:
        result_key = combine_key(key[A], key[B], vertex_kind)
        result = add_or_reuse_current(result_key)
        add_interaction(left=A, right=B, result=result, vertex=vertex_kind)

for retained helicity ancestry:
    add_amplitude(left=current_with_Z_and_gluons, right=anti_current)

prune_dead_currents_after_helicity_filter()
```

At runtime, pyAmpliCol mirrors the Fortran table layout with contiguous arrays:

$$C_{s,i,c} \equiv \texttt{current\_rows}[s,i,c], \qquad T_{s,v,c} \equiv \texttt{interaction\_rows}[s,v,c],$$

where s is the phase-space sample in the current batch, i is a current id, v is an interaction id, and c is a component index. External wavefunctions fill source-current rows once per sample. Bounded evaluator blocks then fill interactions and derived currents stage by stage:

```
fill_source_currents(current_rows, momenta, helicity_sources)

for stage in current_size_layers:
    state = ParamBuilder.pack(current_rows, interaction_rows, momenta)
    interaction_rows[:, stage.vertices] = vertex_eval[stage](state)
    current_rows[:, stage.currents]     = current_eval[stage](state)

amps = amplitude_eval(ParamBuilder.pack(current_rows, momenta))
me2  = normalization * sum(weight[h] * abs(amps[h])**2 for h in retained_h)
```

This is why the evaluator-only benchmark is meaningful: the expensive recursion is inside compiled evaluator calls, while Python only performs source wavefunction construction, parameter packing, and table transfers.

3 Spenso Tensor Networks and `hep_lib`

Spenso is used to express Lorentz, spinor, auxiliary-tensor, and eventually color contractions in a representation-aware way. A tensor-network expression contains named tensors with typed slots:

$$\mathcal{V}_{\mu\nu}{}^{\rho}(p_A, p_B)\, A^{\mu}\, B^{\nu}, \qquad \text{or in code: } \texttt{V(m("a"),m("b"),m("out")) * A(m("a")) * B(m("b"))}.$$

The expression is not expanded. It is a compact product of tensor factors with shared representation labels. The library `hep_lib` is the dictionary that tells Spenso what each tensor name means: dense model tensors, metric tensors, momentum-dependent propagators, and parametric rank-one tensors representing runtime inputs.

```
library = model.build_tensor_library()      # returns TensorLibrary.hep_lib_atom()
builder = ParamBuilder()
mink = Representation.mink(4)

builder.register_rank1_tensor(
    library,
    tensor_name="current_A",
    representation=mink,
    head=("stage", "A"),
    length=4,
    role="block_current",
)
builder.register_rank1_tensor(
    library,
    tensor_name="p_A",
    representation=mink,
    head=("stage", "pA"),
    length=4,
    role="block_momentum",
    real_valued=True,
)

raw = V3g(mink("a"), mink("b"), mink("out"), p_A="p_A", p_B="p_B") \
    * TensorName("current_A")(mink("a")).to_expression() \
    * TensorName("current_B")(mink("b")).to_expression()

network = TensorNetwork(raw, library)
network.execute(library=library)
out_tensor = network.result_tensor(library)
evaluator = out_tensor.evaluator(params=builder.parameter_symbols())
```

The important point is that `hep_lib` stays fixed during the execution. New graph currents are not registered as permanent physics tensors. Instead, the current values are runtime parameters for a bounded block. Spenso executes the factorized tensor network, contracts the representation indices, and returns a parametric output tensor. Symbolica then compiles that parametric tensor to an evaluator. For high multiplicity, pyAmpliCol interleaves this process: after a current or current-size layer is contracted, its numerical output is written back into the current table and becomes an input parameter for later blocks. This avoids one giant scalar expression and keeps the DAG close to AmpliCol's execution model.

The current NLC/full-colour implementation keeps the kinematic DAG colour-ordered and applies a sparse colour matrix to the amplitude vector. For open-line sectors, including arbitrary many quark lines with gluons distributed on the lines, pyAmpliCol contracts the product of left colour strings with the conjugated right strings into closed fundamental traces. Those small colour trace expressions are simplified separately from the kinematic DAG; NLC is obtained by keeping the leading and first $1/N_c^2$-suppressed powers, while full colour keeps the complete sparse metric. If a later implementation puts explicit colour tensors inside a Spenso block, colour simplification still belongs before Spenso execution:

$$\texttt{raw} \leftarrow \texttt{idenso.simplify\_color(raw)}, \qquad \text{then build } \texttt{TensorNetwork(raw, hep\_lib)}.$$

For the present NLC/full-colour milestone, colour algebra is confined to the small colour-basis overlap problem and the Rusticol final contraction, not to expanded kinematic expressions. Fortran AmpliCol is the direct reference for zero, one and two quark-line colour matrices; beyond that, pyAmpliCol's generic open-line metric is validated through raw-amplitude probes and internal colour consistency checks.

4 Example: $d\bar{d} \to Zgg$

This section spells out one concrete DAG so that the split between compiled current contractions and Rust-interpreted scheduling is explicit. The user-level process is

$$d(p_1)\,\bar{d}(p_2) \to Z(p_3)\,g(p_4)\,g(p_5).$$

pyAmpliCol stores the process in the all-outgoing convention. Thus labels $1,\dots,5$ carry the fields

$$\bar{d}_1, \quad d_2, \quad Z_3, \quad g_4, \quad g_5,$$

with momenta $-p_1$, $-p_2$, p_3, p_4, and p_5, respectively. A label set such as $\{2,4,5\}$ is represented by $L = (2,4,5)$ and by the mask

$$m(2,4,5) = 2^{2-1} + 2^{4-1} + 2^{5-1} = 26.$$

The momentum mask is the same bit pattern in this example, and it tells Rusticol which stored momentum sum must be passed to the next compiled evaluator.

For one leading-color sector, the colored ordered labels are

$$O = (2,4,5,1),$$

with the color-singlet Z_3 allowed to attach anywhere on the quark line. The color engine may also provide the sector $O = (2,5,4,1)$. Nothing in the DAG builder special-cases this as a "$Z + 2g$" process: source currents are ordinary external-leg currents and all later currents are discovered from model vertices.

Source current indices

For a fixed retained source helicity choice, the source entries look as follows. The bit b_i denotes the unique helicity-source bit assigned to that external state.

source	current	mask	index payload
S_1	$\mathcal{J}_{\bar{d}}(\{1\})$	1	$(\mathrm{pid} = -1, L = (1), H = b_1, \chi, S, F = (-1), Q, C_{\bar{q}}, P = 1)$
S_2	$\mathcal{J}_{d}(\{2\})$	2	$(\mathrm{pid} = 1, L = (2), H = b_2, \chi, S, F = (1), Q, C_q, P = 2)$
S_3	$\epsilon_Z(\{3\})$	4	$(\mathrm{pid} = 23, L = (3), H = b_3, S = h_Z, C_{\mathrm{singlet}}, P = 4)$
S_4	$\epsilon_g(\{4\})$	8	$(\mathrm{pid} = 21, L = (4), H = b_4, S = h_4, C_g, P = 8)$
S_5	$\epsilon_g(\{5\})$	16	$(\mathrm{pid} = 21, L = (5), H = b_5, S = h_5, C_g, P = 16)$

The actual stored index also contains the exact leading-color sector id, line-group metadata, and the current dimension. For this example the quark and anti-quark source currents have Weyl-spinor components, gluons and the Z have Lorentz-vector components, and auxiliary tensor currents have the antisymmetric tensor components required by the cubicized four-gluon rule.

What each compiled stage computes

The generic stage compiler groups interactions by the size of the result current. For each group it emits a multi-output Symbolica evaluator. That evaluator is compiled once and receives only parent-current components, momentum sums, and real couplings as parameters. It returns the components of all result currents in the stage.

For the sector $O = (2,4,5,1)$, the schematic stage contents are:

$$E_2: \quad \mathcal{J}_g(4,5) = P_g(p_{45})\,\mathcal{V}_{3g}\big(S_4, S_5; p_4, p_5\big), \tag{4}$$

$$\mathcal{J}_T(4,5) = \mathcal{V}_{ggT}\big(S_4, S_5\big), \tag{5}$$

$$\mathcal{J}_d(2,4) = P_q(p_{24})\,\mathcal{V}_{qgq}\big(S_2, S_4; p_2, p_4\big), \tag{6}$$

$$\mathcal{J}_d(2,3) = P_q(p_{23})\,\mathcal{V}_{qZq}\big(S_2, S_3; p_2, p_3\big), \tag{7}$$

plus the analogous allowed currents for the other color sector and for the anti-quark side. The notation E_2 means a compiled evaluator for all two-label result currents, not a Python function that loops over these formulas component by component.

The next stage consumes the rows written by E_2:

$$E_3: \quad \mathcal{J}_d(2,4,5) = P_q(p_{245}) \left[\mathcal{V}_{qgq}\big(\mathcal{J}_d(2,4), S_5\big) + \mathcal{V}_{qgq}\big(S_2, \mathcal{J}_g(4,5)\big)\right], \tag{8}$$

$$\mathcal{J}_d(2,3,4) = P_q(p_{234}) \left[\mathcal{V}_{qZq}\big(\mathcal{J}_d(2,4), S_3\big) + \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3), S_4\big)\right]. \tag{9}$$

The stage compiler does not expand these expressions into diagrams. It collects all interaction records whose output index is the same current and compiles the sum into the output components of that current. If two different splits produce the same current index, their contributions are accumulated inside the same compiled stage output.

The four-label stage then builds the endpoint current used by the closure:

$$E_4: \quad \mathcal{J}_d(2,3,4,5) = P_q(p_{2345}) \sum_{\text{compatible splits}} \mathcal{V}\big(\mathcal{J}_{\text{left}}, \mathcal{J}_{\text{right}}\big). \tag{10}$$

The sum includes, for example, gluon-first and Z-insertion contributions such as

$$\mathcal{V}_{qZq}\big(\mathcal{J}_d(2,4,5), S_3\big), \qquad \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3,4), S_5\big), \qquad \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3), \mathcal{J}_g(4,5)\big).$$

Those are still local vertex applications selected by the model and color engine. The result is one stored current row, identified by the full CurrentIndex and by its value-slot range in the artifact.

Finally, a compiled amplitude evaluator returns raw complex roots:

$$E_{\text{amp}}: \quad R_a = \mathcal{V}_{\text{close}}\big(\mathcal{J}_d(2,3,4,5), S_1\big), \tag{11}$$

with one output for each retained root. Roots with the same $(H, \text{color sector}, O)$ coherent-group key are summed before squaring. The reported leading-color matrix element is

$$|\mathcal{M}|^2_{\text{LC}} = \mathcal{N}_{\text{AC}} \sum_{\text{groups } g} w_g \left| \sum_{a \in g} R_a \right|^2. \tag{12}$$

The same example as a DAG

A stripped-down adjacency-list representation of this example is:

```
sources interpreted by Rusticol:
  S1 = J(pid=-1, labels={1}, H=b1, color=qbar-line)
  S2 = J(pid= 1, labels={2}, H=b2, color=q-line)
  S3 = J(pid=23, labels={3}, H=b3, color=singlet)
  S4 = J(pid=21, labels={4}, H=b4, color=line gluon)
  S5 = J(pid=21, labels={5}, H=b5, color=line gluon)

compiled stage E2, subset_size=2:
  I0: S4 + S5 -- V3g, Pg --> J(pid=21, labels={4,5})
  I1: S4 + S5 -- VggT    --> J(aux=T, labels={4,5})
  I2: S2 + S4 -- Vqgq,Pq --> J(pid=1, labels={2,4})
  I3: S2 + S3 -- VqZq,Pq --> J(pid=1, labels={2,3})

compiled stage E3, subset_size=3:
  I4: J(2,4) + S5     -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
  I5: S2     + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
  I6: J(2,4) + S3     -- VqZq,Pq --> J(pid=1, labels={2,3,4})
  I7: J(2,3) + S4     -- Vqgq,Pq --> J(pid=1, labels={2,3,4})

compiled stage E4, subset_size=4:
  I8: J(2,4,5) + S3     -- VqZq,Pq --> J(pid=1, labels={2,3,4,5})
  I9: J(2,3,4) + S5     -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})
  I10:J(2,3)   + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})

compiled amplitude stage:
  A0: J(pid=1, labels={2,3,4,5}) + S1 -- close --> R0
```

This listing is schematic: the real artifact contains all retained helicity ancestries, both leading-color sectors, source dimensions, output component ranges, momentum-slot ids, couplings, and color weights. The important point is the division of labor. The evaluator E_s performs the tensor contractions and the algebraic sum over interactions in stage s. Rusticol interprets the DAG schedule: fill sources, compute momentum sums, call E_2, scatter its outputs into current storage, call E_3, scatter again, and so on until E_{amp} and the final coherent square-sum.

5 Rusticol: the Implemented Rust Runtime

The previous staged pyAmpliCol runtime proved that the shared-current DAG could match Fortran AmpliCol numerically, but its Python loop still moved current tables between many evaluator calls. The current implementation keeps Python as the process generator and validation driver, but moves the hot eager-DAG sweep into a PyO3 extension named `rusticol`. The generated artifact is now a self-contained process directory used by both runtimes:

```
rt_py = pyamplicol.load_process(process_dir, runtime="python")
values_py = rt_py.evaluate(momenta)

import rusticol
rt = rusticol.Runtime.load(process_dir)
values = rt.evaluate(momenta)
profile = rt.profile(momenta)
```

The Python and Rust runtimes read the same `process_manifest.json`, the same Symbolica evaluator manifests, and the same validation momenta. The Python path is kept as a transparent reference implementation. The Rust path is the performance path.

A process directory contains four classes of data. First, it records process and model metadata: external PDG order, momentum conventions, particle and vertex tables, couplings, masses, symmetry factors, and leading-color normalization. Second, it stores the shared-current DAG: source currents, current ids, dimensions, interaction stages, output slots, retained amplitude records, helicity weights, and color factors. Third, it stores the compiled Symbolica evaluator artifacts for each stage and for the final amplitude/raw-sum stage. Fourth, it contains standalone validation assets: `validation_momenta.json` with decimal-string momenta and a `check_standalone.py` script that imports only `rusticol`, plus `numpy` when available for the optimized f64 input path.

```
# Generated by pyAmpliCol.
process_dir/
  process_manifest.json
  manifest.json
  validation_momenta.json
  check_standalone.py
  compiled/              # C++/ASM shared libraries when present
  ... evaluator states ...
```

`rusticol.Runtime.load` validates the schema, checks that the current process family is supported, loads every Symbolica evaluator, computes direct state offsets for stage outputs, allocates reusable scratch buffers, and builds a fixed execution plan. The optimized f64 path accepts NumPy arrays with shape `(batch, n_external, 4)`. For each batch, Rust fills source wavefunctions and momentum sums, executes stage evaluators in topological order, writes the returned current components into the contiguous state table, executes the amplitude/raw-sum evaluator, applies the Fortran-matched normalization, and returns a NumPy array of matrix elements.

```
for point in batch {
    fill_source_currents(state_row, point);
    fill_momentum_sums(state_row, point);
}

for stage in stages {
    evaluated = stage.evaluator.evaluate_batch(state);
    assign_stage_outputs(state, evaluated, stage.output_offsets);
}

raw = amplitude_stage.evaluate_raw_sums(state);
me2 = normalization * raw;
```

The `profile` API reports the same split used in the benchmarks: source fill, momentum setup, stage evaluator time, output assignment, amplitude evaluator time, final reduction, total wall time, and memory snapshots where the platform exposes them.

The precision API follows Symbolica's model. `evaluate` and `evaluate_with_prec(..., 16)` use the specialized native f64 path. `evaluate_with_prec(..., 32)` maps the serialized exact evaluator state to Symbolica's `DoubleFloat` coefficients. Other positive precisions map to arbitrary-precision `Float` evaluators. The high-precision routes share the same stage orchestration code through generic Rust traits, while the f64 route remains specialized and avoids high-precision allocation overhead.

The artifact schema is process-generic. The staged execution format no longer needs a Rust-side process-family branch: each process directory supplies the source layout, current slots, stage records, amplitude roots, coherent groups, and evaluator artifacts needed by Rusticol. If a model vertex or color rule is not implemented yet, generation fails with an explicit unsupported-kernel diagnostic rather than falling back to a hidden family path.

6 Generic LC Performance Matrix

Each cell compares generated-library AmpliCol against pyAmpliCol staged-DAG JIT and staged-DAG C++ O3 for the same final-state multiplicity n. In each cell, the upper line is generation time; the lower line is wall runtime per phase-space point. The left slot is the generated-library AmpliCol reference value, the middle slot is pyAmpliCol JIT, and the right slot is pyAmpliCol C++ O3. Runtime multipliers are shown as `(wall|core)`. When no AmpliCol reference exists but pyAmpliCol ran, the pyAmpliCol slots show absolute generation time and parenthesized `wall|core` runtime in microseconds. Colored ratios are pyAmpliCol/AmpliCol: green below one, orange below two, red otherwise. When rows are generated with `-validate`, pyAmpliCol JIT and C++ O3 values are compared to the same-point AmpliCol value with a relative tolerance of 10^{-8}.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8}.

<table>
<tr><th>ID</th><th>base process</th><th colspan="3">n=1</th><th colspan="3">n=2</th><th colspan="3">n=3</th></tr>
<tr><td>1</td><td>$d\bar{d} \to Z + (n-1)g$</td><td>2.34 s
0.023 us</td><td>(x0.087)
(x5.59|x0.948)</td><td>(x0.335)
(x5.62|x1.11)</td><td>2.273 s
0.128 us</td><td>(x0.121)
(x2.68|x1.38)</td><td>(x0.59)
(x2.58|x1.40)</td><td>2.657 s
0.45 us</td><td>(x0.131)
(x1.94|x1.38)</td><td>(x0.937)
(x1.93|x1.36)</td></tr>
<tr><td>2</td><td>$u\bar{d} \to W^+ + (n-1)g$</td><td>2.171 s
0.0129 us</td><td>(x0.081)
(x7.12|x0.851)</td><td>(x0.339)
(x7.59|x1.04)</td><td>2.226 s
0.0752 us</td><td>(x0.088)
(x2.80|x1.19)</td><td>(x0.509)
(x2.95|x1.28)</td><td>2.364 s
0.26 us</td><td>(x0.099)
(x2.10|x1.36)</td><td>(x0.82)
(x2.11|x1.38)</td></tr>
<tr><td>3</td><td>$d\bar{d} \to e^+e^- + (n-2)g$</td><td>N/A</td><td></td><td></td><td>2.225 s
0.0659 us</td><td>(x0.083)
(x2.57|x0.997)</td><td>(x0.507)
(x2.64|x1.05)</td><td>2.43 s
0.175 us</td><td>(x0.091)
(x2.19|x1.25)</td><td>(x0.708)
(x2.17|x1.26)</td></tr>
<tr><td>4</td><td>$u\bar{d} \to e^+\nu_e + (n-2)g$</td><td>N/A</td><td></td><td></td><td>2.206 s
0.032 us</td><td>(x0.085)
(x3.47|x0.749)</td><td>(x0.491)
(x3.36|x0.717)</td><td>2.356 s
0.0632 us</td><td>(x0.084)
(x2.99|x1.02)</td><td>(x0.622)
(x3.04|x1.11)</td></tr>
<tr><td>5</td><td>$d\bar{d} \to ZZ + (n-2)g$</td><td>N/A</td><td></td><td></td><td>2.248 s
0.165 us</td><td>(x0.095)
(x2.43|x1.26)</td><td>(x0.621)
(x2.50|x1.37)</td><td>2.475 s
0.729 us</td><td>(x0.135)
(x1.75|x1.29)</td><td>(x1.27)
(x1.72|x1.26)</td></tr>
<tr><td>6</td><td>$gg \to t\bar{t} + (n-2)g$</td><td>N/A</td><td></td><td></td><td>2.324 s
0.0998 us</td><td>(x0.087)
(x3.87|x1.98)</td><td>(x0.573)
(x4.04|x2.19)</td><td>2.759 s
0.695 us</td><td>(x0.105)
(x1.88|x1.40)</td><td>(x1.18)
(x1.96|x1.48)</td></tr>
<tr><td>7</td><td>$d\bar{d} \to t\bar{t} + (n-2)g$</td><td>N/A</td><td></td><td></td><td>2.244 s
0.0504 us</td><td>(x0.1)
(x4.10|x1.10)</td><td>(x0.501)
(x4.28|x1.32)</td><td>2.407 s
0.246 us</td><td>(x0.095)
(x2.43|x1.50)</td><td>(x0.833)
(x2.56|x1.61)</td></tr>
<tr><td>8</td><td>$gg \to gg + (n-2)g$</td><td>N/A</td><td></td><td></td><td>2.279 s
0.178 us</td><td>(x0.087)
(x1.78|x1.05)</td><td>(x0.575)
(x1.86|x1.14)</td><td>2.454 s
0.901 us</td><td>(x0.114)
(x1.83|x1.54)</td><td>(x1.72)
(x1.85|x1.57)</td></tr>
<tr><td>9</td><td>$d\bar{d} \to ZZZ + (n-3)g$</td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td><td>2.846 s
1.592 us</td><td>(x0.179)
(x1.56|x1.22)</td><td>(x2.01)
(x1.47|x1.13)</td></tr>
<tr><td>10</td><td>$d\bar{d} \to e^+e^-ZH + (n-4)g$</td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td></tr>
<tr><td>11</td><td>$d\bar{d} \to t\bar{t}ZH + (n-4)g$</td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td></tr>
<tr><td>12</td><td>$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$</td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td></tr>
<tr><td>13</td><td>$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$</td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td></tr>
<tr><td>14</td><td>$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$</td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td><td>N/A</td><td></td><td></td></tr>
</table>

Generic LC Performance Matrix (continued)

ID	base process	n=4		
1	$d\bar{d} \to Z + (n-1)g$	2.686 s	(x0.184)	(x2.32)
		1.479 us	(x1.72|x1.41)	(x1.71|x1.40)
2	$u\bar{d} \to W^+ + (n-1)g$	2.53 s	(x0.159)	(x1.67)
		0.915 us	(x1.84|x1.48)	(x1.85|x1.49)
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.593 s	(x0.115)	(x1.11)
		0.464 us	(x1.84|x1.33)	(x1.86|x1.34)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.534 s	(x0.096)	(x0.816)
		0.165 us	(x2.45|x1.43)	(x2.49|x1.49)
5	$d\bar{d} \to ZZ + (n-2)g$	2.728 s	(x0.232)	(x2.97)
		2.203 us	(x1.62|x1.34)	(x1.53|x1.23)
6	$gg \to t\bar{t} + (n-2)g$	4.07 s	(x0.129)	(x2.55)
		2.482 us	(x1.75|x1.46)	(x1.78|x1.48)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	3.025 s	(x0.122)	(x1.76)
		0.924 us	(x2.22|x1.78)	(x2.22|x1.79)
8	$gg \to gg + (n-2)g$	2.847 s	(x0.187)	(x5.26)
		3.363 us	(x1.92|x1.71)	(x1.91|x1.69)
9	$d\bar{d} \to ZZZ + (n-3)g$	4.002 s	(x0.314)	(x4.45)
		5.04 us	(x1.45|x1.21)	(x1.30|x1.05)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	4.791 s	(x0.088)	(x0.938)
		0.777 us	(x2.20|x1.73)	(x2.15|x1.68)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	3.856 s	(x0.113)	(x1.35)
		1.107 us	(x1.92|x1.54)	(x1.82|x1.44)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.446 s	(x0.093)	(x1.02)
		1.519 us	(x1.13|x0.921)	(x1.07|x0.866)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	2.554 s	(x0.082)	(x0.728)
		0.182 us	(x1.93|x1.05)	(x2.02|x1.12)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		

Matrix run settings. The matrix table is generated with one uniform benchmark setup. The full refresh command is:

```
pyAmpliCol/dependencies/.venv/bin/python pyAmpliCol/docs/result_matrix.py --color-accuracy lc --data
    /Users/vjhirsch/HEP_programs/AmpliCol/pyAmpliCol/docs/result_matrix_data.json --table
    /Users/vjhirsch/HEP_programs/AmpliCol/pyAmpliCol/docs/result_matrix_table.tex --generate-data 1-9 --show-data
    1-9 --time-limit 900 --target-runtime 1 --amplicol-points 10000 --jobs 4 --n-cores 4 --process-workers 1
    --columns-per-table 3
```

Partial refreshes use the same settings with `-base-process`, `-skip-jit`, `-skip-cpp-o3`, `-process-ids`, or `-skip-amplicol` added only to restrict which cells are updated. PyAmpliCol-only refreshes may additionally use `-process-workers 5`; this is intentionally disabled when the AmpliCol reference is refreshed because that path uses shared top-level generated-library files. AmpliCol reference timings use the generated-library sequence `amplicol_generate -library=create`, `make -j4 amplicol_generate_library`, and a direct generated-library benchmark using `-library=use`; they do not use integration-driver timings. pyAmpliCol rows use the generic LC staged-DAG artifact, Rusticol double precision, JIT or C++ O3 Symbolica evaluators as labelled, and `time-process` with a one-second target runtime. The `-time-limit` setting applies only to C++ O3 generation; AmpliCol and JIT rows are left uncapped.

7 Generic NLC Performance Matrix

Each cell compares raw generated-library AmpliCol colour probes against pyAmpliCol staged-DAG JIT and staged-DAG C++ O3 for the same final-state multiplicity n. In each cell, the upper line is generation time; the lower line is wall runtime per phase-space point. The left slot is the generated-library AmpliCol reference value, the middle slot is pyAmpliCol JIT, and the right slot is pyAmpliCol C++ O3. Runtime multipliers are shown as `(wall|core)`. When no AmpliCol reference exists but pyAmpliCol ran, the pyAmpliCol slots show absolute generation time and parenthesized `wall|core` runtime in microseconds. For NLC/full-colour tables, AmpliCol reference values come from the dedicated raw generated-library colour driver `amplicol_color_library_probe`. Colored ratios are pyAmpliCol/AmpliCol: green below one, orange below two, red otherwise. When rows are generated with `-validate`, pyAmpliCol JIT and C++ O3 values are compared to the same-point AmpliCol value with a relative tolerance of 10^{-8}.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8}.

ID	base process	n=1			n=2			n=3		
1	$d\bar{d} \to Z + (n-1)g$	2.224 s	(x0.083)	(x0.347)	2.265 s	(x0.081)	(x0.575)	2.389 s	(x0.113)	(x1.49)
		11.46 us	(x0.013|x0.002)	(x0.013|x0.003)	25.33 us	(x0.013|x0.006)	(x0.013|x0.007)	93.76 us	(x0.018|x0.013)	(x0.018|x0.013)
2	$u\bar{d} \to W^+ + (n-1)g$	2.191 s	(x0.079)	(x0.388)	2.257 s	(x0.079)	(x0.495)	2.342 s	(x0.096)	(x1.02)
		11.15 us	(x0.01|x0.001)	(x0.01|x0.002)	23.25 us	(x0.009|x0.004)	(x0.01|x0.004)	73.55 us	(x0.014|x0.01)	(x0.014|x0.009)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A			2.253 s	(x0.078)	(x0.478)	2.324 s	(x0.081)	(x0.693)
					17.12 us	(x0.01|x0.004)	(x0.01|x0.004)	39.66 us	(x0.009|x0.005)	(x0.009|x0.005)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A			2.192 s	(x0.081)	(x0.46)	2.294 s	(x0.077)	(x0.603)
					15.38 us	(x0.007|x0.001)	(x0.007|x0.002)	34.55 us	(x0.005|x0.002)	(x0.006|x0.002)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A			2.328 s	(x0.082)	(x0.579)	2.346 s	(x0.109)	(x1.25)
					39.84 us	(x0.01|x0.005)	(x0.011|x0.006)	120 us	(x0.011|x0.008)	(x0.01|x0.008)
6	$gg \to t\bar{t} + (n-2)g$	N/A			2.345 s	(x0.086)	(x0.768)	2.639 s	(x0.213)	(x6.70)
					21.93 us	(x0.037|x0.022)	(x0.038|x0.023)	194 us	(x0.088|x0.078)	(x0.087|x0.076)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A			2.232 s	(x0.083)	(x0.547)	2.893 s	(x0.109)	(x1.58)
					18.47 us	(x0.023|x0.009)	(x0.024|x0.01)	78.64 us	(x0.032|x0.022)	(x0.032|x0.021)
8	$gg \to gg + (n-2)g$	N/A			2.413 s	(x0.096)	(x1.59)	3.617 s	(x0.474)	(x25.1)
					53.82 us	(x0.032|x0.025)	(x0.033|x0.026)	1.36e+03 us	(x0.037|x0.033)	(x0.033|x0.03)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A			N/A			3.794 s	(x0.103)	(x1.64)
								289 us	(x0.009|x0.007)	(x0.009|x0.007)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A			N/A			N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A			N/A			N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A			N/A			N/A		
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A			N/A			N/A		
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A		

Generic NLC Performance Matrix (continued)

ID	base process	n=4		
1	$d\bar{d} \to Z + (n-1)g$	2.75 s	(x0.402)	(x18.1)
		1e+03 us	(x0.038|x0.035)	(x0.045|x0.041)
2	$u\bar{d} \to W^+ + (n-1)g$	2.696 s	(x0.264)	(x8.36)
		642 us	(x0.023|x0.02)	(x0.026|x0.022)
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.433 s	(x0.136)	(x1.72)
		170 us	(x0.01|x0.008)	(x0.01|x0.008)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.322 s	(x0.103)	(x1.08)
		108 us	(x0.006|x0.004)	(x0.007|x0.005)
5	$d\bar{d} \to ZZ + (n-2)g$	2.506 s	(x0.274)	(x7.15)
		809 us	(x0.01|x0.008)	(x0.009|x0.007)
6	$gg \to t\bar{t} + (n-2)g$	5.038 s	(x1.38)	timeout
		4.57e+03 us	(x0.096|x0.091)	(N/A|N/A)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	4.594 s	(x0.338)	(x27.6)
		967 us	(x0.071|x0.067)	(x0.078|x0.072)
8	$gg \to gg + (n-2)g$	16.95 s	(x2.38)	timeout
		4.55e+04 us	(x0.016|x0.014)	(N/A|N/A)
9	$d\bar{d} \to ZZZ + (n-3)g$	3.823 s	(x0.157)	(x4.72)
		1.33e+03 us	(x0.005|x0.004)	(x0.005|x0.004)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	5.085 s	(x0.051)	(x0.899)
		107 us	(x0.016|x0.012)	(x0.016|x0.013)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	3.194 s	(x0.12)	(x3.37)
		197 us	(x0.023|x0.018)	(x0.023|x0.017)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.904 s	(x0.053)	(x0.995)
		233 us	(x0.007|x0.006)	(x0.007|x0.006)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A	0.384 s	5.647 s
		N/A	(2.86|1.76)	(3.06|1.88)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		

Matrix run settings. The matrix table is generated with one uniform benchmark setup. The full refresh command is:

```
pyAmpliCol/dependencies/.venv/bin/python pyAmpliCol/docs/result_matrix.py --color-accuracy nlc --data
     /Users/vjhirsch/HEP_programs/AmpliCol/pyAmpliCol/docs/result_matrix_nlc_data.json --table
     /Users/vjhirsch/HEP_programs/AmpliCol/pyAmpliCol/docs/result_matrix_nlc_table.tex --generate-data 1-9
     --show-data 1-9 --time-limit 900 --target-runtime 1 --amplicol-points 10000 --jobs 4 --n-cores 4
     --process-workers 1 --columns-per-table 3
```

Partial refreshes use the same settings with `-base-process`, `-skip-jit`, `-skip-cpp-o3`, `-process-ids`, or `-skip-amplicol` added only to restrict which cells are updated. PyAmpliCol-only refreshes may additionally use `-process-workers 5`; this is intentionally disabled when the AmpliCol reference is refreshed because that path uses shared top-level generated-library files. AmpliCol reference timings for this colour table use the raw generated-library sequence `amplicol_generate -library=create-raw`, `make -j4 amplicol_generate_library`, and the dedicated `amplicol_color_library_probe`. The raw library is created before LC helicity merging so that NLC/full colour-order interferences are preserved. pyAmpliCol rows use the generic staged-DAG artifact, Rusticol double precision, JIT or C++ O3 Symbolica evaluators as labelled, and `time-process` with a one-second target runtime. The `-time-limit` setting applies only to C++ O3 generation; AmpliCol and JIT rows are left uncapped.

Fortran colour-reference limitations. The following cells are generated and evaluated by pyAmpliCol, but have no direct AmpliCol NLC/full-colour matrix reference because the current Fortran `init_col` reference path stops above two quark lines: $d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$, $n = 4$. Their pyAmpliCol slots are shown as absolute timings.

C++ O3 compile-budget gaps. The following cells have validated AmpliCol and pyAmpliCol JIT rows but no completed C++ O3 row; their C++ slot is a localized performance-generation compile-budget gap, not a physics-validation failure: $gg \to t\bar{t} + (n-2)g$, $n = 4$: timed out; $gg \to gg + (n-2)g$, $n = 4$: timed out.

8 Generic Full-Colour Performance Matrix

Each cell compares raw generated-library AmpliCol colour probes against pyAmpliCol staged-DAG JIT and staged-DAG C++ O3 for the same final-state multiplicity n. In each cell, the upper line is generation time; the lower line is wall runtime per phase-space point. The left slot is the generated-library AmpliCol reference value, the middle slot is pyAmpliCol JIT, and the right slot is pyAmpliCol C++ O3. Runtime multipliers are shown as `(wall|core)`. When no AmpliCol reference exists but pyAmpliCol ran, the pyAmpliCol slots show absolute generation time and parenthesized `wall|core` runtime in microseconds. For NLC/full-colour tables, AmpliCol reference values come from the dedicated raw generated-library colour driver `amplicol_color_library_probe`. Colored ratios are pyAmpliCol/AmpliCol: green below one, orange below two, red otherwise. When rows are generated with `-validate`, pyAmpliCol JIT and C++ O3 values are compared to the same-point AmpliCol value with a relative tolerance of 10^{-8}.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8}.

ID	base process	n=1			n=2			n=3		
1	$d\bar{d} \to Z + (n-1)g$	2.231 s 11.02 us	(x0.088) (x0.013|x0.002)	(x0.331) (x0.013|x0.003)	2.296 s 25.6 us	(x0.094) (x0.013|x0.006)	(x0.563) (x0.013|x0.007)	2.985 s 98.8 us	(x0.123) (x0.021|x0.014)	(x1.44) (x0.019|x0.014)
2	$u\bar{d} \to W^+ + (n-1)g$	2.168 s 11.56 us	(x0.077) (x0.01|x0.001)	(x0.36) (x0.01|x0.002)	2.225 s 23.79 us	(x0.081) (x0.009|x0.004)	(x0.509) (x0.009|x0.004)	3.106 s 76.39 us	(x0.111) (x0.015|x0.01)	(x0.934) (x0.014|x0.01)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A			2.218 s 17.03 us	(x0.083) (x0.01|x0.004)	(x0.482) (x0.01|x0.004)	2.963 s 42.55 us	(x0.093) (x0.009|x0.005)	(x0.696) (x0.01|x0.005)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A			2.317 s 15.57 us	(x0.091) (x0.007|x0.001)	(x0.448) (x0.007|x0.001)	3.295 s 35.42 us	(x0.082) (x0.006|x0.002)	(x0.528) (x0.006|x0.002)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A			2.281 s 39.86 us	(x0.119) (x0.01|x0.005)	(x0.636) (x0.011|x0.006)	2.946 s 125 us	(x0.12) (x0.012|x0.008)	(x1.18) (x0.011|x0.008)
6	$gg \to t\bar{t} + (n-2)g$	N/A			2.345 s 21.98 us	(x0.088) (x0.037|x0.022)	(x0.791) (x0.038|x0.023)	3.371 s 203 us	(x0.207) (x0.097|x0.082)	(x5.98) (x0.098|x0.083)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A			2.411 s 19.5 us	(x0.083) (x0.022|x0.009)	(x0.524) (x0.023|x0.01)	3.227 s 82.75 us	(x0.111) (x0.034|x0.023)	(x1.59) (x0.039|x0.023)
8	$gg \to gg + (n-2)g$	N/A			2.571 s 55.38 us	(x0.113) (x0.032|x0.025)	(x1.58) (x0.034|x0.026)	4.395 s 1.42e+03 us	(x0.428) (x0.037|x0.032)	(x20.4) (x0.033|x0.029)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A			N/A			3.658 s 306 us	(x0.155) (x0.009|x0.007)	(x1.72) (x0.008|x0.006)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A			N/A			N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A			N/A			N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A			N/A			N/A		
13	$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$	N/A			N/A			N/A		
14	$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$	N/A			N/A			N/A		

Generic Full-Colour Performance Matrix (continued)

ID	base process	n=4		
1	$d\bar{d} \to Z + (n-1)g$	2.796 s	(x0.374)	(x19.1)
		1.02e+03 us	(x0.038|x0.035)	(x0.046|x0.042)
2	$u\bar{d} \to W^+ + (n-1)g$	2.674 s	(x0.254)	(x8.92)
		655 us	(x0.023|x0.019)	(x0.029|x0.025)
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.384 s	(x0.108)	(x1.84)
		164 us	(x0.01|x0.008)	(x0.011|x0.008)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.297 s	(x0.11)	(x1.11)
		110 us	(x0.007|x0.004)	(x0.007|x0.005)
5	$d\bar{d} \to ZZ + (n-2)g$	2.517 s	(x0.251)	(x7.01)
		801 us	(x0.009|x0.007)	(x0.01|x0.008)
6	$gg \to t\bar{t} + (n-2)g$	5.174 s	(x1.44)	timeout
		4.69e+03 us	(x0.079|x0.073)	(N/A|N/A)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	4.662 s	(x0.341)	(x28.9)
		968 us	(x0.088|x0.08)	(x0.081|x0.075)
8	$gg \to gg + (n-2)g$	16.8 s	(x2.93)	timeout
		4.71e+04 us	(x0.017|x0.013)	(N/A|N/A)
9	$d\bar{d} \to ZZZ + (n-3)g$	3.383 s	(x0.175)	(x5.40)
		1.3e+03 us	(x0.005|x0.004)	(x0.006|x0.005)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	4.686 s	(x0.059)	(x0.989)
		107 us	(x0.016|x0.012)	(x0.016|x0.013)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	3.161 s	(x0.117)	(x3.26)
		195 us	(x0.023|x0.018)	(x0.022|x0.018)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.957 s	(x0.051)	(x0.99)
		227 us	(x0.007|x0.006)	(x0.008|x0.006)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A	0.446 s	5.529 s
		N/A	(3.18|1.88)	(3.73|2.07)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		

Matrix run settings. The matrix table is generated with one uniform benchmark setup. The full refresh command is:

```
pyAmpliCol/dependencies/.venv/bin/python pyAmpliCol/docs/result_matrix.py --color-accuracy full --data
     /Users/vjhirsch/HEP_programs/AmpliCol/pyAmpliCol/docs/result_matrix_full_data.json --table
     /Users/vjhirsch/HEP_programs/AmpliCol/pyAmpliCol/docs/result_matrix_full_table.tex --generate-data 1-9
     --show-data 1-9 --time-limit 900 --target-runtime 1 --amplicol-points 10000 --jobs 4 --n-cores 4
     --process-workers 1 --columns-per-table 3
```

Partial refreshes use the same settings with `-base-process`, `-skip-jit`, `-skip-cpp-o3`, `-process-ids`, or `-skip-amplicol` added only to restrict which cells are updated. PyAmpliCol-only refreshes may additionally use `-process-workers 5`; this is intentionally disabled when the AmpliCol reference is refreshed because that path uses shared top-level generated-library files. AmpliCol reference timings for this colour table use the raw generated-library sequence `amplicol_generate -library=create-raw`, `make -j4 amplicol_generate_library`, and the dedicated `amplicol_color_library_probe`. The raw library is created before LC helicity merging so that NLC/full colour-order interferences are preserved. pyAmpliCol rows use the generic staged-DAG artifact, Rusticol double precision, JIT or C++ O3 Symbolica evaluators as labelled, and `time-process` with a one-second target runtime. The `-time-limit` setting applies only to C++ O3 generation; AmpliCol and JIT rows are left uncapped.

Fortran colour-reference limitations. The following cells are generated and evaluated by pyAmpliCol, but have no direct AmpliCol NLC/full-colour matrix reference because the current Fortran `init_col` reference path stops above two quark lines: $d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$, $n = 4$. Their pyAmpliCol slots are shown as absolute timings.

C++ O3 compile-budget gaps. The following cells have validated AmpliCol and pyAmpliCol JIT rows but no completed C++ O3 row; their C++ slot is a localized performance-generation compile-budget gap, not a physics-validation failure: $gg \to t\bar{t} + (n-2)g$, $n = 4$: timed out; $gg \to gg + (n-2)g$, $n = 4$: timed out.

Archived low-multiplicity validation fixture. The $n \leq 4$ entries of the LC, NLC and full-colour process matrices are also stored as a compact integration-test fixture in `tests/fixtures/result_matrix_references.json`. The fixture contains the process string, colour accuracy, deterministic phase-space point, Fortran AmpliCol reference value, pyAmpliCol JIT value, tolerance and relative difference for each validated cell. It is checked by `tests/integration/test_result_matrix_references.py` without requiring a local Fortran AmpliCol run. To refresh the fixture from the matrix caches, run `scripts/refresh_result_matrix_fixtures.py`; with `-refresh-caches`, the script first reruns the documented `result_matrix.py` validation workflow against Fortran AmpliCol.

9 Performance Results

Tables 7 and 8 report the current benchmark state for $d\bar{d} \to Z + ng$, $n = 1, \ldots, 9$. All successful pyAmpliCol rows have been validated against Fortran AmpliCol with maximum relative differences typically of order 10^{-13}. The Fortran reference rows use the generated-library workflow `-library=create`, compiled `amplicol_generate_library`, and probe evaluation with `-library=use`. Timing values are shown in monospace font. Green, orange, and red multiplier text means faster than Fortran AmpliCol, slower but below a factor two, and slower by a factor two or more. The blue rows are the Fortran AmpliCol references. The green highlighted rows are the current pyAmpliCol production route, `rusticol` with C++ O3 staged evaluators. The dependency installer currently uses upstream Symbolica `dev` with the pyAmpliCol local patches and pins SymJIT to `2.19.3` at commit `dee77e14c8bf9c8d5304dbda2e393c52ee3e4cd4`. This fixes the historical AArch64 complex-JIT crash captured by `MRE_symjit_bug_new.py`; staged-DAG JIT rows refreshed after this dependency update are noted in the tables below.

Table 7: Staged/Rusticol performance summary for $n = 1, \ldots, 4$.

n route	setup	gen [s]	wall [us/pt]	eval [us/pt]	notes
1 reference	**AmpliCol**	2.22	N/A	0.182	**Fortran AmpliCol; with** `-library=use`
1 Python staged	pyAmpliCol - JIT	0.016 (x0.01)	13.34(29) (x73.10)	0.607(17) (x3.33)	JIT; batch=16
1 Python staged	pyAmpliCol - C++	0.774 (x0.35)	13.01(12) (x71.29)	0.521(10) (x2.85)	C++; O3; chunk=None; batch=16
1 Python staged	pyAmpliCol - ASM	0.560 (x0.25)	14.03(30) (x76.88)	0.912(18) (x5.00)	ASM; O3; chunk=None; batch=16
1 Rusticol	**pyAmpliCol - Rusticol**	0.984 (x0.44)	0.412(1) (x2.26)	0.1938(2) (x1.06)	Rusticol PyO3; C++ O3 process artifact; cached NumPy input; reusable stage scratch; single-chunk direct output; batch=1024
2 reference	**AmpliCol**	2.17	N/A	0.520	**Fortran AmpliCol; with** `-library=use`
2 Python staged	pyAmpliCol - JIT	0.066 (x0.03)	16.15(15) (x31.04)	1.61(2) (x3.09)	JIT; batch=16
2 Python staged	pyAmpliCol - C++	1.86 (x0.86)	16.05(14) (x30.85)	1.16(2) (x2.23)	C++; O3; chunk=None; batch=16
2 Python staged	pyAmpliCol - ASM	0.940 (x0.43)	16.48(20) (x31.67)	2.16(2) (x4.15)	ASM; O3; chunk=None; batch=16
2 Rusticol	**pyAmpliCol - Rusticol**	2.00 (x0.92)	0.984(1) (x1.89)	0.653(1) (x1.26)	Rusticol PyO3; C++ O3 process artifact; cached NumPy input; reusable stage scratch; single-chunk direct output; batch=1024
3 reference	**AmpliCol**	2.29	N/A	1.61	**Fortran AmpliCol; with** `-library=use`
3 Python staged	pyAmpliCol - JIT	0.281 (x0.12)	22.67(42) (x14.08)	4.72(5) (x2.93)	JIT; batch=16
3 Python staged	pyAmpliCol - C++	5.69 (x2.48)	20.90(40) (x12.98)	3.06(4) (x1.90)	C++; O3; chunk=None; batch=16
3 Python staged	pyAmpliCol - ASM	1.52 (x0.66)	23.87(45) (x14.83)	6.17(8) (x3.83)	ASM; O3; chunk=None; batch=16
3 Rusticol	**pyAmpliCol - Rusticol**	5.48 (x2.39)	2.783(13) (x1.73)	2.177(10) (x1.35)	Rusticol PyO3; C++ O3 process artifact; cached NumPy input; reusable stage scratch; batch=128
4 reference	**AmpliCol**	2.65	N/A	5.11	**Fortran AmpliCol; with** `-library=use`
4 Python staged	pyAmpliCol - JIT	0.431 (x0.16)	14.23(1) (x2.78)	11.20(1) (x2.19)	JIT; batch=16; last successful JIT refresh
4 Python staged	pyAmpliCol - C++	20.7 (x7.80)	27.78(13) (x5.43)	7.75(6) (x1.52)	C++; O3; chunk=None; batch=16
4 Python staged	pyAmpliCol - ASM	2.04 (x0.77)	36.30(54) (x7.10)	15.80(15) (x3.09)	ASM; O3; chunk=None; batch=16
4 Rusticol	**pyAmpliCol - Rusticol**	19.9 (x7.50)	7.23(7) (x1.41)	6.10(6) (x1.19)	Rusticol PyO3; C++ O3 process artifact; cached NumPy input; reusable stage scratch; batch=128

Table 8: Staged/Rusticol performance summary for $n = 5, \ldots, 9$.

n	route	setup	gen [s]	wall [us/pt]	eval [us/pt]	notes
5	reference	**AmpliCol**	3.26	N/A	13.9	**Fortran AmpliCol; with** `-library=use`
5	Python staged	pyAmpliCol - JIT	0.991 (x0.30)	44.33(5) (x3.19)	40.56(4) (x2.92)	JIT; batch=16; last successful JIT refresh
5	Python staged	pyAmpliCol - C++	54.3 (x16.67)	45.06(48) (x3.25)	20.83(26) (x1.50)	C++; O3; chunk=64; batch=128; chunk compile workers=10; two-stage save/load
5	Python staged	pyAmpliCol - C++ (O2)	50.7 (x15.57)	51.04(87) (x3.68)	23.82(42) (x1.72)	C++; O2; chunk=64; batch=16
5	Python staged	pyAmpliCol - ASM	10.2 (x3.13)	75.69(88) (x5.45)	48.74(72) (x3.51)	ASM; O3; chunk=64; batch=16
5	Rusticol	**pyAmpliCol - Rusticol**	50.2 (x15.41)	19.63(28) (x1.41)	17.50(25) (x1.26)	Rusticol PyO3; C++ O3; chunk=64; cached NumPy input; reusable stage scratch; batch=128; two-stage save/load
6	reference	**AmpliCol**	4.66	N/A	37.3	**Fortran AmpliCol; with** `-library=use`
6	Python staged	pyAmpliCol - JIT	2.96 (x0.64)	163.0(5) (x4.37)	157.0(5) (x4.21)	JIT; batch=16; last successful JIT refresh
6	Python staged	pyAmpliCol - C++	151 (x32.44)	94.79(73) (x2.54)	56.36(49) (x1.51)	C++; O3; chunk=64; batch=128; chunk compile workers=10; two-stage save/load
6	Python staged	pyAmpliCol - C++ (O2)	140 (x30.07)	108.4(1.8) (x2.91)	68.5(1.6) (x1.84)	C++; O2; chunk=64; batch=16
6	Python staged	pyAmpliCol - ASM	21.4 (x4.60)	183.7(3.3) (x4.93)	141.2(2.7) (x3.79)	ASM; O3; chunk=64; batch=16
6	Rusticol	**pyAmpliCol - Rusticol**	141 (x30.29)	55.63(26) (x1.49)	50.12(23) (x1.34)	Rusticol PyO3; C++ O3; chunk=64; cached NumPy input; reusable stage scratch; batch=128; two-stage save/load
7	reference	**AmpliCol**	9.28	N/A	95.2	**Fortran AmpliCol; with** `-library=use`
7	Python staged	pyAmpliCol - JIT	8.85 (x0.95)	536(2) (x5.63)	525(2) (x5.52)	JIT; batch=16; last successful JIT refresh
7	Python staged	pyAmpliCol - C++	417 (x44.92)	259.7(1.4) (x2.73)	194.7(1.2) (x2.05)	C++; O3; chunk=96; batch=128; chunk compile workers=10; two-stage save/load
7	Python staged	pyAmpliCol - C++ (O1)	219 (x23.59)	421.6(2.4) (x4.43)	348.9(1.9) (x3.67)	C++; O1; chunk=96; batch=16
7	Python staged	pyAmpliCol - ASM	36.2 (x3.90)	384.4(2.8) (x4.04)	320.2(2.4) (x3.36)	ASM; O3; chunk=96; batch=16
7	Rusticol	**pyAmpliCol - Rusticol**	391 (x42.12)	197.8(4) (x2.08)	186.1(4) (x1.95)	Rusticol PyO3; C++ O3; chunk=96; cached NumPy input; reusable stage scratch; batch=128; two-stage save/load
8	reference	**AmpliCol**	26.1	N/A	234	**Fortran AmpliCol; with** `-library=use`
8	Python staged	pyAmpliCol - JIT	42.3 (x1.62)	1783(4) (x7.61)	1763(4) (x7.52)	JIT; batch=16; last successful JIT refresh
8	Python staged	pyAmpliCol - C++	1.06e3 (x40.69)	642.6(2.1) (x2.74)	491.8(1.4) (x2.10)	C++; O3; chunk=96; batch=128; chunk compile workers=10; two-stage save/load
8	Python staged	pyAmpliCol - C++ (O1)	577 (x22.15)	997.2(3.3) (x4.26)	885.3(2.9) (x3.78)	C++; O1; chunk=96; batch=16
8	Python staged	pyAmpliCol - ASM	85.6 (x3.29)	1117.0(4.6) (x4.77)	1004.9(4.3) (x4.29)	ASM; O3; chunk=96; batch=16
8	Rusticol	**pyAmpliCol - Rusticol**	1.02e3 (x39.15)	487.7(9) (x2.08)	464.4(6) (x1.98)	Rusticol PyO3; C++ O3; chunk=96; cached NumPy input; reusable stage scratch; batch=128; two-stage save/load
9	reference	**AmpliCol**	39.4	N/A	567	**Fortran AmpliCol; with** `-library=use`
9	Python staged	pyAmpliCol - JIT	80.8 (x2.05)	6144(17) (x10.84)	6102(15) (x10.77)	JIT; batch=16; last successful JIT refresh
9	Python staged	pyAmpliCol - C++	2.83e3 (x71.80)	1504.9(11.7) (x2.66)	1252.7(9.6) (x2.21)	C++; O3; chunk=96; batch=128; chunk compile workers=10; two-stage save/load
9	Python staged	pyAmpliCol - ASM	349 (x8.85)	2688.2(5.5) (x4.74)	2459.1(4.5) (x4.34)	ASM; O3; chunk=64; batch=16
9	Rusticol	**pyAmpliCol - Rusticol**	2.83e3 (x71.80)	1164.8(9) (x2.06)	1120.9(1.0) (x1.98)	Rusticol PyO3; C++ O3; chunk=96; cached NumPy input; reusable stage scratch; batch=128; repackaged from existing C++ O3 artifact in 103 s; two-stage save/load

10 Why This Matches AmpliCol but Remains Python-Friendly

The Python layer owns graph construction, process metadata, deterministic phase-space steering, artifact generation, evaluator caching, and validation against AmpliCol. It does not perform the hot current recursion component by component in production. The two production execution routes are now:

$$\begin{aligned} \text{Python staged route:} &\quad \text{source fill} \rightarrow \text{bounded stage evaluators} \rightarrow \text{amplitude/raw-sum evaluator}, \\ \text{Rusticol route:} &\quad \text{one PyO3 call} \rightarrow \text{Rust source fill and staged sweep} \rightarrow \text{amplitude/raw-sum evaluator}. \end{aligned}$$

Thus pyAmpliCol keeps AmpliCol's essential optimization - shared currents across helicity parents and a topological current-table sweep - while replacing generated Fortran subroutines by saved Spenso/Symbolica evaluator artifacts. The `rusticol` path removes the remaining Python orchestration cost by loading the same process directory and executing the sweep in Rust.

The old fully monolithic compiled-DAG route is now deprecated and kept only as reference code. Its alias-based design was useful for exploring whole-graph compilation, but the production target is the staged DAG: Python generation plus a Rust eager-DAG runtime over saved Symbolica stage evaluators. This keeps the runtime schedule close to AmpliCol's current-table library while leaving the graph compiler model-driven and process-generic.